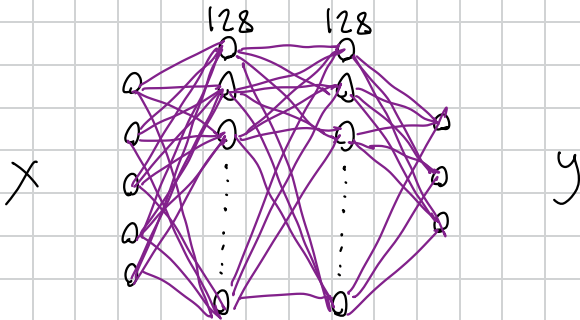


Neural Network IK 3 link

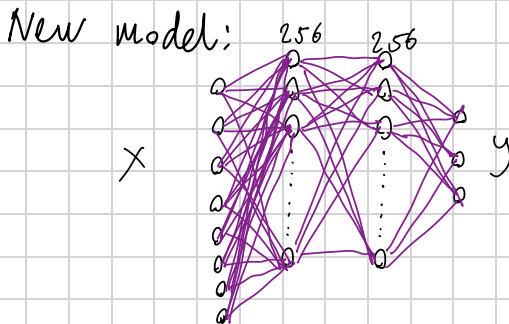


Input x : target x , target y , θ_1 , θ_2 , θ_3

output y : $\Delta \theta_1$, $\Delta \theta_2$, $\Delta \theta_3$

Ideas and explanation behind structure and training remains the same as 2-link, check other file for more info.

This NN model design yielded poor results. 13.2% Success, 0.442 mean error, and 19.4 ms mean time.



Receives target x , target y , θ_1 , θ_2 , θ_3 , but converts the target position into normalized Cartesian error and the angles into sine and cosine features

Why the Improved 3-Link Neural Policy Worked Better

The main improvement came from redesigning the learning problem, not only from making the network larger.

1. Better input features

The model still receives the target position and current joint angles. Inside the model, the target is converted into Cartesian error:

```
error_x = target_x - current_x  
error_y = target_y - current_y
```

This directly tells the network how far and in which direction the arm must move. Dividing by 6 keeps the errors roughly between -1 and 1. The angles are represented with $\sin(\theta)$ and $\cos(\theta)$, avoiding the jump between $-\pi$ and π .

2. Trajectory-based data

The first dataset used unrelated random states. The improved dataset saves every step of full DLS trajectories:

```
random start -> DLS update -> save state -> repeat
```

This trains the policy on the same types of states it meets during a complete rollout.

3. Leakage-free splits

Training, validation, and test data are generated separately.

This stops nearby steps from the same trajectory appearing in different splits and making the results look better than they really are.

4. Predicting updates

A 3-link arm can reach one target with many different joint configurations. The model therefore predicts:

```
[delta_theta1, delta_theta2, delta_theta3]
```

These small updates are applied repeatedly until the target is reached.

5. Task-aware loss

The loss checks both whether the model copies DLS and whether the predicted step actually moves the arm correctly:

```
total loss = DLS imitation loss + 0.1 x end-effector loss
```

This makes the training objective closer to the real IK task.

6. Bounded updates

Each update is limited to a norm of 0.5.

This prevents one poor prediction from causing a large and unstable movement.

7. Larger model

The architecture changed from 5 -> 128 -> 128 -> 3 to 8 -> 256 -> 256 -> 256 -> 3.

The extra capacity helped, but the better data, features, loss, and bounded updates were the main reasons for the improvement.

Result: Success improved from 13.2% to 95.9%, while mean error fell from 0.4415 to 0.0400.